



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

Names:	Molar, Jabez C. Oabel, Edmarc Justin C.	Date:	August 30, 2026
Program:	BS CPE		

- A. Learning Outcomes:**
- At the end of this laboratory, students should be able to:**
1. Differentiate Strings, Lists, Tuples, and Dictionaries.
 2. Apply appropriate Python data structures in software design.
 3. Develop a simple console-based application using collections.
 4. Demonstrate teamwork through collaborative software development.
 5. Analyze the advantages and limitations of different data structures.

B. Introduction:

Python provides several built-in data structures used to organize and manage data efficiently.

1. Strings

Strings are used to store text data such as:

```
name = "Juan Dela Cruz"
course = "BS Computer Engineering"
```

- Use Strings For:
- Student Name
 - Course
 - Labels and Output Messages

2. Lists

Lists are used to store multiple subjects.

```
subjects = [
"Programming",
"Calculus",
"Physics"
]
```

- Use Lists For:
- Subject Management
 - Displaying Subject Information

3. Tuples

Tuples store fixed information that should not be changed.

```
student_id = (2026, 1001)
```

- Use Tuples For:
- Student ID
 - Year and Student Number

4. Dictionary

Store complete record:

```
student = {
"Name": name,
"Course": course,
"Student ID": student_id,
"Subjects": subjects
}
```

Used as the primary data container.

Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

C. Laboratory Scenario:

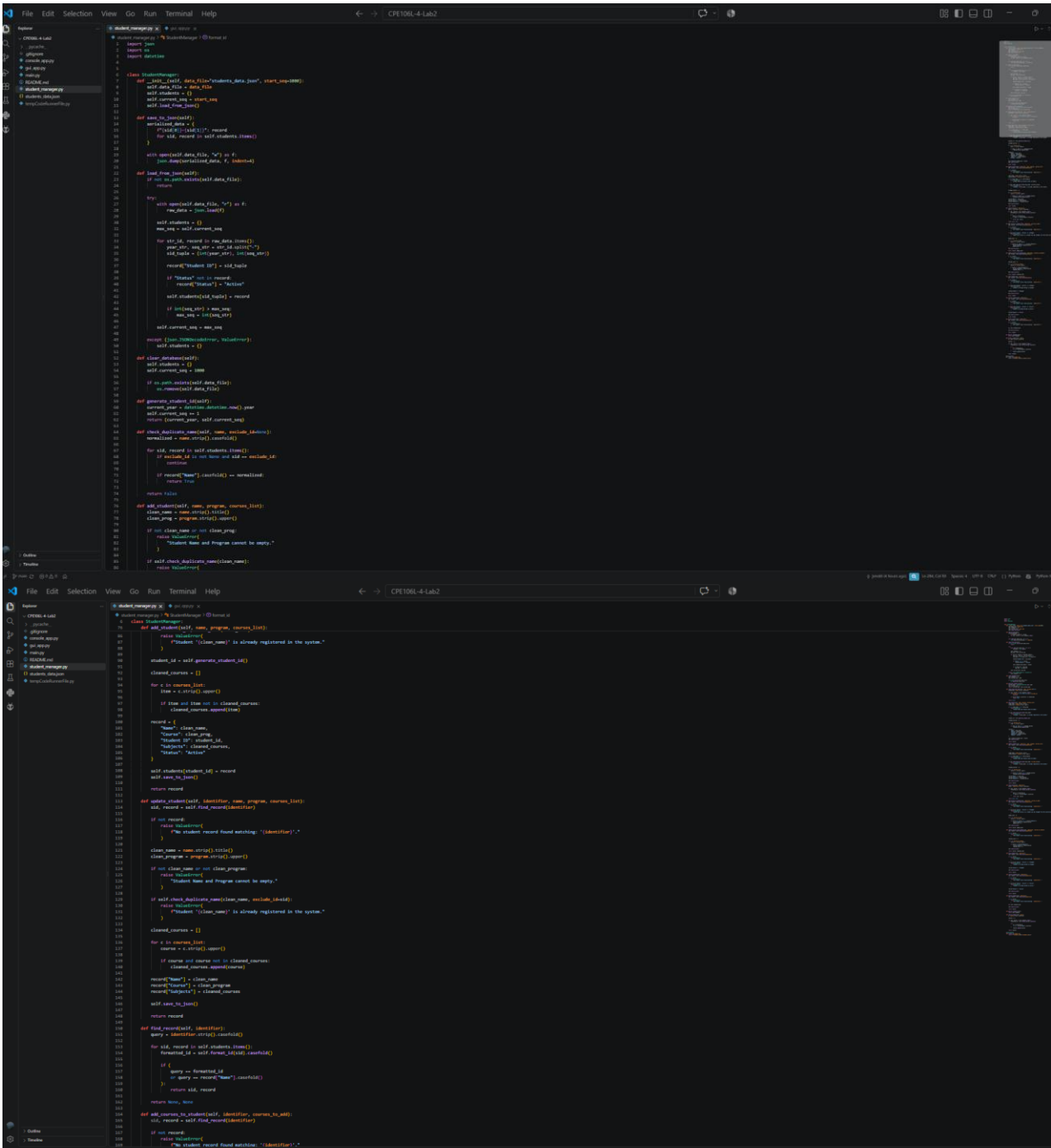
A university needs a simple **Student Information Management System**. 2 Members
The system should:

1. Store student names.
2. Manage enrolled subjects.
3. Generate a unique student identifier.
4. Save student details using dictionaries.

(Please include the following: **1.** Screenshots of the code **2.** Screenshots of the output)

Screenshots of Code (Name of Members) – MOLAR, OABEL

student_manager.py



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

console_app.py

```
1 from console_app import StudentManager
2
3 def register_all():
4     print("\n" * 4)
5     print("===== REGISTRATION MODE =====")
6     print("\n" * 4)
7
8     name = input("Enter Name: ").strip()
9     program = input("Enter Program: ").strip()
10    raw_courses = input("Enter Courses (comma-separated): ").strip()
11    courses = [c.strip() for c in raw_courses.split(",") if c.strip()]
12
13    try:
14        record = manager.add_student(name, program, courses)
15        formatted_id = manager.format_id(record["Student ID"])
16        print(f"[SUCCESS] Record initialized. Assigned ID: {formatted_id}")
17    except ValueError as err:
18        print(f"[ERROR] {err}")
19
20 def manage_courses_all():
21    print("\n" * 4)
22    print("===== MANAGE COURSES =====")
23    print("\n" * 4)
24
25    identifier = input("Enter Student ID or Name: ").strip()
26    old_record = manager.find_record(identifier)
27
28    if not record:
29        print(f"[ERROR] No record found matching: '{identifier}'")
30        return
31
32    print(f"Student Found : {record['Name']} ({manager.format_id(record['Student ID'])})")
33    print(f"Current Courses : {', '.join(record['Subjects']) if record['Subjects'] else 'None'}")
34
35    print("\nChoose Action:")
36    print("1. Add Courses")
37    print("2. Drop / Delete Courses")
38    action = input("Select (1-2): ").strip()
39
40    if action == "1":
41        raw_input = input("Enter Courses to Add (comma-separated): ").strip()
42        target_list = [c.strip() for c in raw_input.split(",") if c.strip()]
43
44        updated_count = manager.add_courses_to_student(identifier, target_list)
45        print(f"[SUCCESS] Added {updated_count} new course(s).")
46        print(f"Updated Record: {', '.join(updated['Subjects']) if updated['Subjects'] else 'None'}")
47        manager.display_all()
48        print(f"[SUCCESS] {action}")
49
50    elif action == "2":
51        raw_input = input("Enter Courses to Drop (comma-separated): ").strip()
52        target_list = [c.strip() for c in raw_input.split(",") if c.strip()]
53
54        updated_count = manager.remove_courses_from_student(identifier, target_list)
55        print(f"[SUCCESS] Removed {updated_count} course(s).")
56        print(f"Updated Record: {', '.join(updated['Subjects']) if updated['Subjects'] else 'None'}")
57        manager.display_all()
58        print(f"[SUCCESS] {action}")
59
60    else:
61        print(f"[INVALID] Operation cancelled.")
62
63 def display_all():
64    print("\n" * 4)
65    print("===== ACTION STUDENT DIRECTORY =====")
66    print("\n" * 4)
67
68    records = manager.get_all_students()
69
70    if not records:
71        print("No student records currently stored.")
72        return
73
74    for record in records:
75        print(f"Student ID: {manager.format_id(record['Student ID'])}")
76        print(f"Name: {record['Name']}")
77        print(f"Program: {record['Program']}")
78        print(f"Courses: {record['Subjects']} (Serialized -> {courses_str})")
79        print("\n" * 4)
80
81 def search_all():
82    print("\n" * 4)
83    print("===== SEARCH DIRECTORY =====")
84    print("\n" * 4)
85
86    query = input("Enter Student ID or Name: ").strip()
87    matches = manager.query_students(query)
88
89    if not matches:
90        print(f"[NOTICE] No matching student record located.")
91        return
92
93    for data in matches:
94        formatted_id = manager.format_id(data["Student ID"])
95        courses_str = ", ".join(data["Subjects"]) if data["Subjects"] else "None"
96        print(f"[MATCH FOUND]")
97        print(f"ID Code : {formatted_id}")
98        print(f"Name : {data['Name']}")
99        print(f"Program : {data['Program']}")
100       print(f"Courses : {data['Subjects']}")
101
102 def clear_all():
103    print("\n" * 4)
104    print("===== RESET DATABASE FILE =====")
105    print("\n" * 4)
106
107    confirm = input("Are you sure you want to clear all stored JSON records? (y/n): ").strip().lower()
108    if confirm == "y":
109        manager.clear_database()
110        print(f"[SUCCESS] Database cleared and JSON file reset.")
111    else:
112        print(f"[CANCELLED] Database reset aborted.")
113
114 while True:
115    print("\n" * 4)
116    print("===== UNIVERSITY ACADEMIC RECORDS SYSTEM =====")
117    print("\n" * 4)
118
119    print("1. Register New Student")
120    print("2. Manage Enrolled Courses (Add/Drop)")
121    print("3. Display All Student Records")
122    print("4. Query Student Directory")
123    print("5. Reset / Clear Database")
124    print("6. Return to Main Interface")
125
126    choice = input("Select operation (1-6): ").strip()
127
128    if choice == "1":
129        register_all()
130    elif choice == "2":
131        manage_courses_all()
132    elif choice == "3":
133        display_all()
134    elif choice == "4":
135        search_all()
136    elif choice == "5":
137        clear_all()
138    elif choice == "6":
139        print("Exiting console interface...")
140        break
141
142 print(f"[INVALID] Please select a valid option (1-6).")
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

Screenshots of Output

```
=====
CPE106L - APPLICATION ENTRY LAUNCHER
=====
1. Launch Console-Based System
2. Launch Graphical User Interface (GUI)
3. Reset All JSON Data
4. Exit Application

Enter choice (1-4): 1

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 3

=====
ACTIVE STUDENT DIRECTORY
=====

ID Code   : 2026-1001
Name      : Edmarc Justin C. Oabel
Program   : BS COMPUTER ENGINEERING
Courses   : 7 Enrolled -> CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L, IE103-1
-----

ID Code   : 2026-1002
Name      : Jabez C. Molar
Program   : BS COMPUTER ENGINEERING
Courses   : 7 Enrolled -> CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-4, CPE117-4
-----

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 1
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

```
[CANCELLED] Database reset aborted.

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 2

=====
MANAGE ENROLLED COURSES
=====
Enter Student ID or Name: Justin De Leon

Student Found   : Justin De Leon (2026-1003)
Current Courses : CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L, IE103-1

Choose Action:
1. Add Course(s)
2. Drop / Delete Course(s)
Select (1-2): 2
Enter Courses to Drop (comma-separated): IE103-1

[SUCCESS] Dropped 1 course(s).
Updated Enrolled: CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 3

=====
ACTIVE STUDENT DIRECTORY
=====

ID Code   : 2026-1001
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

```

ID Code : 2026-1001
Name : Edmarc Justin C. Oabel
Program : BS COMPUTER ENGINEERING
Courses : 7 Enrolled -> CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L, IE103-1
-----

ID Code : 2026-1002
Name : Jabez C. Molar
Program : BS COMPUTER ENGINEERING
Courses : 7 Enrolled -> CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-4, CPE117-4
-----

ID Code : 2026-1003
Name : Justin De Leon
Program : BS COMPUTER ENGINEERING
Courses : 6 Enrolled -> CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L
-----

#####
UNIVERSITY ACADEMIC RECORDS SYSTEM
#####
1. Register New Student
2. Manage Enrolled Courses (Add/Drop)
3. Display All Student Records
4. Query Student Directory
5. Reset / Clear Database
6. Return to Main Launcher

Select operation (1-6): 6
Exiting console interface...

=====
CPE106L - APPLICATION ENTRY LAUNCHER
=====
1. Launch Console-Based System
2. Launch Graphical User Interface (GUI)
3. Reset All JSON Data
4. Exit Application

Enter choice (1-4): █

```

Laboratory Report 2
Strings, Lists, Tuples, and Dictionaries

- 5. Basic GUI using Tkinter. No need to be creative yet, just create a functional GUI and embed the system.
2 Members

(Please include the following: 1. Screenshots of the code 2. Screenshots of the output)

Screenshots of Code (Name of Members) - MOLAR, OABEL

gui_app.py

```
1 def new_gui_manager():
2     import tkinter as tk
3     from tkinter import messagebox, ttk
4     from student_manager import StudentManager
5
6     manager = StudentManager()
7
8     # Colors
9     BG_COLOR = "#f0f0f0"
10    CANV_COLOR = "#ffffff"
11    PRIMARY_COLOR = "#4682b4"
12    SECONDARY_COLOR = "#808080"
13    ACCENT_COLOR = "#808080"
14    TEXT_COLOR = "#000000"
15    MENU_TEXT = "#808080"
16    SUCCESS_COLOR = "#90ee90"
17    DANGER_COLOR = "#ff4500"
18    WARNING_COLOR = "#ffa500"
19
20    def refresh_table():
21        for item in tree.get_children():
22            tree.delete(item)
23
24        for id, record in manager.get_all_students().items():
25            formatted_id = manager.format_id(id)
26            courses = record.get("subjects", [])
27
28            tree.insert(
29                "",
30                tk.END,
31                values=[
32                    formatted_id,
33                    record["name"],
34                    record["course"],
35                    len(courses),
36                    *join(courses) if courses else "None",
37                    record.get("status", "Active")
38                ]
39            )
40
41    def clear_fields():
42        entry_name.delete(0, tk.END)
43        entry_program.delete(0, tk.END)
44        entry_courses.delete(0, tk.END)
45
46        entry_name.focus()
47
48    status_label.config(
49        text="Ready for student registration."
50    )
51
52    def register_student_gui():
53        name = entry_name.get().strip()
54        program = entry_program.get().strip()
55        raw_courses = entry_courses.get().strip()
56
57        courses = []
58        for c in raw_courses.split(","):
59            if c.strip():
60                courses.append(c.strip())
61
62        if not name or not program:
63            messagebox.showwarning(
64                "Incomplete Information",
65                "Please enter both the student's Name and Program."
66            )
67            return
68
69        try:
70            record = manager.add_student(
71                name,
72                program,
73                courses
74            )
75
76            formatted_id = manager.format_id(
77                record["id"]
78            )
79
80            refresh_table()
81            clear_fields()
82
83            status_label.config(
84                text=f"Student registered successfully - ID: {formatted_id}"
85            )
86
87            messagebox.showinfo(
88                "Registration Successful",
89                f"Record initialized successfully.\n"
90                f"Student ID: {formatted_id}\n"
91                f"Name: {record['name']}\n"
92                f"Program: {record['course']}"
93            )
94
95        except ValueError as e:
96            messagebox.showerror(
97                "Registration Error",
98                str(e)
99            )
100
101    def get_selected_student():
102        selected = tree.selection()
103
104        if not selected:
105            messagebox.showwarning(
106                "No Student Selected",
107                "Please select a student from the directory first."
108            )
109            return None
110
111        return tree.item(selected[0], "values")
112
113    def edit_student_gui():
114        values = get_selected_student()
115
116        if not values:
117            return
118
119        student_id = values[0]
```

main.py

Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

```
... main.py X
main.py > main
1 import sys
2 from student_manager import StudentManager
3 from console_app import run_console_app
4 from gui_app import run_gui_app
5
6 def main():
7     manager = StudentManager()
8
9     while True:
10         print("\n=====")
11         print(" CPE106L - APPLICATION ENTRY LAUNCHER ")
12         print("=====")
13         print("1. Launch Console-Based System")
14         print("2. Launch Graphical User Interface (GUI)")
15         print("3. Reset All JSON Data")
16         print("4. Exit Application")
17
18         choice = input("\nEnter choice (1-4): ").strip()
19
20         if choice == "1":
21             run_console_app(manager)
22         elif choice == "2":
23             run_gui_app(manager)
24         elif choice == "3":
25             confirm = input("Reset stored data file? (y/n): ").strip().lower()
26             if confirm == 'y':
27                 manager.clear_database()
28                 print("JSON storage file reset successfully.")
29         elif choice == "4":
30             print("\nShutting down application. Goodbye!")
31             sys.exit(0)
32         else:
33             print("\nInvalid choice. Please input 1, 2, 3, or 4.")
34
35 if __name__ == "__main__":
36     main()
```



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

Screenshots of Output

University Academic Records System

UNIVERSITY ACADEMIC RECORDS SYSTEM

Student Registration & Academic Directory

Student Registration

Enter the student's academic information below.

Student Name

Justin De Leon

UPDATE STUDENT

Program

BS COMPUTER ENGINEERING

CLEAR FIELDS

Courses

CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1L

CANCEL EDIT

Active Student Directory

Registered student records

ID Code	Name	Program	Courses	Enrolled Courses	Status
2026-1001	Edmarc Justin C. Oabel	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Active
2026-1002	Jabez C. Molar	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-	Active
2026-1003	Justin De Leon	BS COMPUTER ENGINEERING	6	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Active

EDIT STUDENT

DROP COURSE

DROP STUDENT

RESTORE STUDENT

DELETE RECORD

Drop Course

Drop Course(s)

Justin De Leon (2026-1003)

Enter course(s) to drop:

Example: CPE106L-4, MATH165

DROP COURSE

Editing student 2026-1003 — modify the fields and click UPDATE STUDENT.

University Academic Records System

UNIVERSITY ACADEMIC RECORDS SYSTEM

Student Registration & Academic Directory

Student Registration

Enter the student's academic information below.

Student Name

REGISTER STUDENT

Program

CLEAR FIELDS

Courses

Active Student Directory

Registered student records

ID Code	Name	Program	Courses	Enrolled Courses	Status
2026-1001	Edmarc Justin C. Oabel	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Active
2026-1002	Jabez C. Molar	BS COMPUTER ENGINEERING	7	CPE106L-4, ECEA101-1, ECEA101-1L, CPE106-4, CPE105-4, CPE121-	Active
2026-1003	Justin De Leon	BS COMPUTER ENGINEERING	6	CPE106L-4, ECEA101-1, CPE117-4, CPE106-4, CPE105-4, ECEA101-1	Dropped

EDIT STUDENT

DROP COURSE

DROP STUDENT

RESTORE STUDENT

DELETE RECORD

Student 2026-1003 has been marked as DROPPED.



Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

D. Findings, Observations, and Comments:

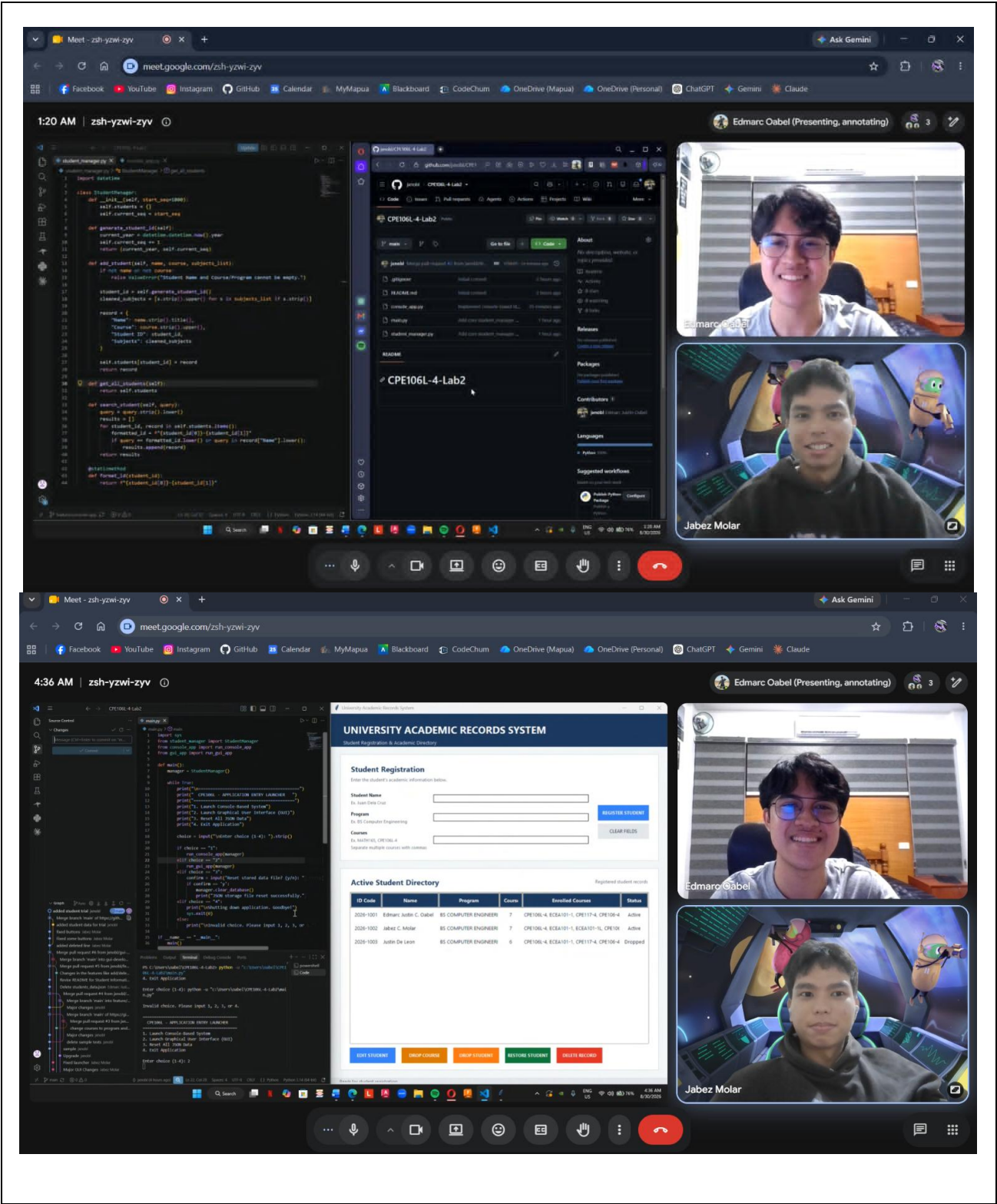
Guide Questions:

1. What are your Group’s realizations in this laboratory?
- 2.

Through this activity, our group realized how GitHub and VS Code can be used for collaboration and software development. We learned how to use push, pull, merge, and branches to manage our code and work together without affecting each other’s work. The activity also taught us the purpose of these features and how they can help organize our projects, track changes, and avoid conflicts. We also realized the importance of communication when working with groupmates, especially when deciding who would work on specific parts of the project. Using branches allowed us to work on our assigned tasks separately and combine our work when it was ready.

Laboratory Report 2
Strings, Lists, Tuples, and Dictionaries

PICTURE/PROOF OF DOING THE ACTIVITY





Laboratory Report 2

Strings, Lists, Tuples, and Dictionaries

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indentation, but inappropriate elements and structures and poor program documentation.	The program had corrected indentation, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indentation, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings, observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings, observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	